

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №28

Тема: Створення алгоритмів впорядкування

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Навчитися застосовувати алгоритми впорядкування для розв'язання прикладних задач: сортування записів за одним і кількома ключами, використання стійкості, користувацьких функцій порівняння, бінарного пошуку у впорядкованих даних, злиття впорядкованих послідовностей, знаходження k-го за величиною елемента та підрахунку інверсій.

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal).

Короткі теоретичні відомості

1. Впорядкування записів. На практиці сортують не окремі числа, а **записи** (кортежі, словники, об'єкти) за значенням **ключа** — деякого поля запису. У Python для цього використовують параметр `key`: `sorted(students, key=lambda s: s["score"])`. Параметр `reverse=True` задає спадний порядок. Функція `key` викликається один раз для кожного елемента.

2. Сортування за кількома ключами. Є два способи: (а) ключ-кортеж — `key=lambda s: (s["group"], -s["score"])` (спочатку за групою, потім за спаданням балу); (б) **послідовне стійке сортування**: спочатку за менш значущим ключем, потім за більш значущим. Другий спосіб працює завдяки **стійкості** — рівні за головним ключем записи зберігають порядок, встановлений попереднім сортуванням.

3. Функція порівняння. Якщо порядок не зводиться до ключа (наприклад, потрібно скласти найбільше число зі списку чисел), використовують `functools.cmp_to_key(cmp)`, де `cmp(a, b)` повертає від'ємне число, 0 або додатне залежно від того, що `a` «менше», «дорівнює» чи «більше» `b`.

4. Бінарний (двійковий) пошук у впорядкованому масиві: порівнюють шуканий елемент із середнім; якщо він менший — продовжують у лівій половині, якщо більший — у правій; складність $O(\log n)$ (для $n = 10^6$ — не більше 20 порівнянь, тоді як лінійний пошук — до 10^6). Умова застосування — масив **впорядкований**. Модуль `bisect` надає `bisect_left` (перша позиція, куди можна вставити елемент) та `bisect_right`; з їх допомогою рахують кількість елементів у діапазоні та підтримують впорядкованість при вставці.

5. Злиття двох впорядкованих послідовностей виконується за $O(n + m)$ двома вказівниками (це основа сортування злиттям); для багатьох послідовностей застосовують `heapq.merge`.

6. Вибірка k-го за величиною елемента (Quickselect). Для медіани чи k-го найменшого не потрібно сортувати весь масив ($O(n \log n)$): після розбиття за опорним елементом (як у QuickSort) продовжують лише ту частину, де знаходиться k-й елемент. Середня складність $O(n)$. Медіана — елемент із індексом $n // 2$ у впорядкованому масиві.

7. Інверсії. Інверсія – пара індексів $i < j$, для якої $a[i] > a[j]$. Кількість інверсій вимірює «невпорядкованість» масиву: 0 для відсортованого, $n(n-1)/2$ для масиву у зворотному порядку. Це також кількість обмінів сусідніх елементів, потрібних бульбашковому сортуванню. Кількість інверсій можна підрахувати за $O(n \log n)$ під час сортування злиттям.

Хід роботи

Завдання 1. Ручне впорядкування записів за кількома ключами

Дано список студентів (прізвище, група, бал): (Іваненко, 2, 88), (Петренко, 1, 95), (Коваль, 2, 95), (Бондар, 1, 88), (Мельник, 2, 76), (Шевчук, 1, 95). Впорядкувати його двома способами.

Порядок виконання:

1. Впорядкуйте записи за групою (за зростанням), а в межах групи — за балом (за спаданням). Запишіть результат.
2. Отримайте той самий результат двома стійкими сортуваннями: спочатку за балом (спадання), потім за групою; запишіть проміжний і кінцевий списки й порівняйте з пунктом 1.
3. Змініть порядок сортувань (спочатку за групою, потім за балом) і поясніть, чому результат змінився (який ключ тепер головний).
4. Для записів із рівним балом у межах групи визначте порядок за алфавітом прізвищ, додавши третій ключ.
5. Запишіть, скільки записів переставляє кожен з двох способів і чому в другому способі важлива стійкість проміжного сортування.

Завдання 2. Ручний бінарний пошук і злиття

Виконати вручну бінарний пошук у впорядкованому масиві та злиття двох впорядкованих масивів.

Порядок виконання:

1. У масиві $A = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]$ знайдіть бінарним пошуком число 23: складіть таблицю «крок — left — right — mid — $A[mid]$ — рішення» та підрахуйте кількість порівнянь.
2. Виконайте пошук числа 40 (його немає в масиві): запишіть, коли пошук припиняється ($left > right$), і знайдіть позицію, на яку це число потрібно вставити, щоб зберегти впорядкованість.
3. Оцініть максимальну кількість порівнянь для масиву з 10 елементів ($\lfloor \log_2 10 \rfloor + 1$) та для 1 000 000 елементів.
4. Злийте впорядковані масиви $B = [1, 4, 9, 15]$ і $C = [2, 3, 10, 11, 20]$ двома вказівниками: складіть таблицю « i — j — $B[i]$ — $C[j]$ — що записано» та результат; порахуйте кількість порівнянь.
5. Знайдіть вручну кількість інверсій у масиві $[3, 1, 4, 2]$ (перелічіть усі пари) і поясніть, скільки обмінів сусідніх елементів потрібно бульбашковому сортуванню.

Завдання 3. Програмна реалізація: впорядкування записів, бінарний пошук, злиття

Реалізувати мовою Python сортування записів за кількома ключами, користувацький порівнювач, бінарний пошук і злиття; перевірити результати ручних трасувань.

Порядок виконання:

1. Подайте список студентів як список словників; відсортуйте його за групою та спаданням балу ключем-кортежем та послідовним стійким сортуванням; переконайтесь, що результати збігаються з ручними.
2. Напишіть функцію `largest_number(nums)`, що складає з натуральних чисел найбільше можливе число (наприклад, `[3, 30, 34, 5, 9] → 9534330`); для цього застосуйте `cmp_to_key` з порівнянням рядків `a+b` та `b+a`.
3. Напишіть функцію `binary_search(a, x)`, що повертає індекс елемента або `-1` та кількість порівнянь; перевірте на пошуку 23 і 40 у масиві A із завдання 2.
4. Напишіть функцію `merge_sorted(b, c)` двома вказівниками та перевірте на масивах із завдання 2 й на порожніх масивах.
5. Напишіть функцію `count_in_range(a, lo, hi)` на основі `bisect_left` і `bisect_right`, що підраховує кількість елементів впорядкованого масиву в діапазоні `[lo, hi]` за $O(\log n)$.

Приклад реалізації мовою Python:

python

```
from functools import cmp_to_key
import bisect

students = [
    {"name": "Іваненко", "group": 2, "score": 88},
    {"name": "Петренко", "group": 1, "score": 95},
    {"name": "Коваль", "group": 2, "score": 95},
    {"name": "Бондар", "group": 1, "score": 88},
    {"name": "Мельник", "group": 2, "score": 76},
    {"name": "Шевчук", "group": 1, "score": 95},
]

by_tuple = sorted(students, key=lambda s: (s["group"], -s["score"], s["name"]))
by_stable = sorted(sorted(students, key=lambda s: s["name"]), key=lambda s: -s["score"])
by_stable = sorted(by_stable, key=lambda s: s["group"])
print("Група Прізвище Бал")
for s in by_tuple:
    print(f"{s['group']:>5} {s['name']:<9} {s['score']:>4}")
print("Два способи збігаються:", by_tuple == by_stable)

def largest_number(nums):
    def cmp(a, b):
        return -1 if a + b > b + a else (1 if a + b < b + a else 0)
    parts = sorted(map(str, nums), key=cmp_to_key(cmp))
    return "".join(parts)

def binary_search(a, x):
    left, right, comparisons = 0, len(a) - 1, 0
    while left <= right:
        mid = (left + right) // 2
        comparisons += 1
        if a[mid] == x:
            return mid, comparisons
        if a[mid] < x:
            left = mid + 1
        else:
            right = mid - 1
    return -1, comparisons
```

```

def merge_sorted(b, c):
    i = j = 0
    result = []
    while i < len(b) and j < len(c):
        if b[i] <= c[j]:
            result.append(b[i])
            i += 1
        else:
            result.append(c[j])
            j += 1
    return result + b[i:] + c[j:]

def count_in_range(a, lo, hi):
    return bisect.bisect_right(a, hi) - bisect.bisect_left(a, lo)

print("\nНайбільше число з [3, 30, 34, 5, 9]:", largest_number([3, 30, 34, 5, 9]))
A = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
print("binary_search(23) =", binary_search(A, 23))
print("binary_search(40) =", binary_search(A, 40), "; місце вставки:",
bisect.bisect_left(A, 40))
print("merge_sorted =", merge_sorted([1, 4, 9, 15], [2, 3, 10, 11, 20]))
print("merge_sorted з порожнім:", merge_sorted([], [7, 8]), merge_sorted([], []))
print("Елементів A у діапазоні [10, 60]:", count_in_range(A, 10, 60))

```

Результат виконання програми:

```

Група  Прізвище  Бал
1  Петренко    95
1  Шевчук     95
1  Бондар      88
2  Коваль      95
2  Іваненко    88
2  Мельник     76
Два способи збігаються: True

Найбільше число з [3, 30, 34, 5, 9]: 9534330
binary_search(23) = (5, 3)
binary_search(40) = (-1, 4) ; місце вставки: 7
merge_sorted = [1, 2, 3, 4, 9, 10, 11, 15, 20]
merge_sorted з порожнім: [7, 8] []
Елементів A у діапазоні [10, 60]: 5

```

Висновок за результатами: ключ-кортеж і послідовне стійке сортування дають однаковий порядок записів, що збігається з ручним результатом: спочатку група 1 (Петренко 95, Шевчук 95, Бондар 88), потім група 2 (Коваль 95, Іваненко 88, Мельник 76). Бінарний пошук знаходить число 23 за 3 порівняння (елемент 40 відсутній, а його місце вставки — індекс 7), тоді як лінійний пошук потребував би 6 порівнянь. Користувацький порівнювач дає найбільше число 9534330 — це типова задача, де звичайне сортування рядків за алфавітом помилилось би.

Завдання 4. Медіана, k-й елемент та підрахунок інверсій

Реалізувати вибірку k-го елемента без повного сортування, порівняти її з сортуванням та підрахувати інверсії під час сортування злиттям.

Порядок виконання:

1. Напишіть функцію `quickselect(a, k)` (k рахуємо з 0), що розбиває масив на елементи менші, рівні та більші за опорний (випадковий) і рекурсивно продовжує в потрібній частині; поверніть значення k-го найменшого.
2. Знайдіть медіану масиву з 1001 випадкового числа за допомогою `quickselect` і перевірте результат порівнянням із `sorted(a)[500]`.

3. Виміряйте час quickselect і повного сортування для $n = 10^5$ та 10^6 (найкращий результат із 3 запусків) і порівняйте; поясніть різницю складностей $O(n)$ та $O(n \log n)$.
4. Напишіть функцію count_inversions(a) на основі сортування злиттям: під час злиття, якщо береться елемент правої частини, до лічильника додається кількість елементів, що залишилися в лівій частині.
5. Перевірте count_inversions на масиві [3, 1, 4, 2] (очікується 3), на відсортованому масиві (0) та на масиві у зворотному порядку з 100 елементів ($n(n-1)/2 = 4950$); порівняйте з підрахунком за визначенням (два вкладені цикли) на випадковому масиві.

Приклад реалізації мовою Python:

python

```
import random
import time

random.seed(7)

def quickselect(a, k):
    pivot = random.choice(a)
    less = [x for x in a if x < pivot]
    equal = [x for x in a if x == pivot]
    greater = [x for x in a if x > pivot]
    if k < len(less):
        return quickselect(less, k)
    if k < len(less) + len(equal):
        return pivot
    return quickselect(greater, k - len(less) - len(equal))

def count_inversions(a):
    if len(a) <= 1:
        return a, 0
    mid = len(a) // 2
    left, inv_l = count_inversions(a[:mid])
    right, inv_r = count_inversions(a[mid:])
    merged, i, j, inv = [], 0, 0, inv_l + inv_r
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
            inv += len(left) - i
    return merged + left[i:] + right[j:], inv

def best_time(func, repeats=3):
    best = float("inf")
    for _ in range(repeats):
        t0 = time.perf_counter()
        func()
        best = min(best, time.perf_counter() - t0)
    return best

data = [random.randint(0, 10**6) for _ in range(1001)]
print("Медіана quickselect:", quickselect(data, 500), "; sorted:", sorted(data)[500])

for n in (10**5, 10**6):
    arr = [random.randint(0, 10**9) for _ in range(n)]
    t_sel = best_time(lambda: quickselect(arr, n // 2))
```

```

t_sort = best_time(lambda: sorted(arr))
print(f"n = {n:>7}: quickselect {t_sel:.4f} c, sorted {t_sort:.4f} c")

print("\nІнверсії [3, 1, 4, 2]:", count_inversions([3, 1, 4, 2])[1])
print("Відсортований масив:", count_inversions(list(range(100)))[1])
print("Зворотний порядок, n = 100:", count_inversions(list(range(100, 0, -1)))[1])
test = [random.randint(0, 50) for _ in range(300)]
brute = sum(1 for i in range(300) for j in range(i + 1, 300) if test[i] > test[j])
print("Випадковий масив: злиттям =", count_inversions(test)[1], ", за визначенням =",
brute)

```

Результат виконання програми:

```

Медіана quickselect: 490456 ; sorted: 490456
n = 100000: quickselect 0.0118 c, sorted 0.0185 c
n = 1000000: quickselect 0.1175 c, sorted 0.2653 c

Інверсії [3, 1, 4, 2]: 3
Відсортований масив: 0
Зворотний порядок, n = 100: 4950
Випадковий масив: злиттям = 21647 , за визначенням = 21647

```

Висновок за результатами: quickselect знаходить медіану без повного сортування, і результат збігається з sorted(a) [500]. Його середня складність $O(n)$ проти $O(n \log n)$ для сортування, тому зі зростанням n виграш збільшується (абсолютні значення часу залежать від комп'ютера). Підрахунок інверсій під час злиття дає ті самі значення, що й прямий перебір усіх пар, але за $O(n \log n)$ замість $O(n^2)$: для масиву у зворотному порядку з 100 елементів – рівно $4950 = n(n-1)/2$, для відсортованого – 0.

Контрольні питання

1. Що таке ключ сортування? Як у Python відсортувати список записів за кількома ключами?
2. Чому послідовне сортування за кількома ключами починають із найменш значущого? Яку роль відіграє стійкість?
3. Для чого використовують cmp_to_key? Наведіть приклад, коли сортування за ключем недостатнє.
4. Опишіть алгоритм бінарного пошуку. Яка його складність і за якої умови його можна застосовувати?
5. Чим відрізняються bisect_left та bisect_right? Як за їх допомогою підрахувати елементи в діапазоні?
6. Як виконується злиття двох впорядкованих масивів? Яка складність цієї операції?
7. Як знайти медіану масиву без повного сортування? Яка середня складність quickselect?
8. Що таке інверсія? Як підрахувати кількість інверсій за $O(n \log n)$ і чому цей показник характеризує «впорядкованість» масиву?

Зміст звіту

1. Тема, мета роботи.
2. Ручні трасування впорядкування, бінарного пошуку та злиття (завдання 1, 2).
3. Тексти програм та результати їх роботи (завдання 3, 4).

4. Висновки.